Design Document

Project Title: Gamified App For Student

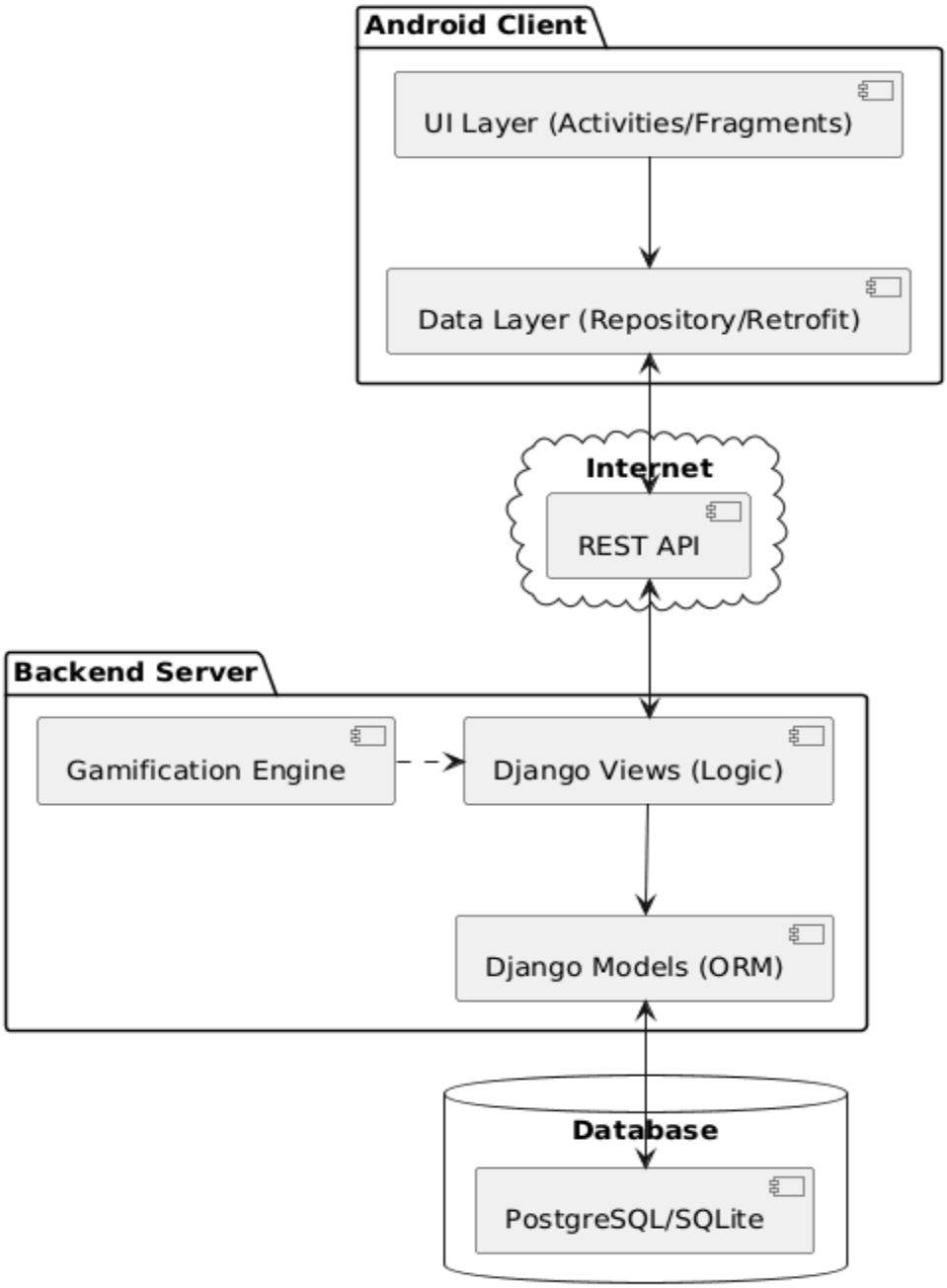
Author: Shuai Jiang

Date: 14/11/2025

1. System Architecture

The system follows a **Decoupled Client-Server Architecture**:

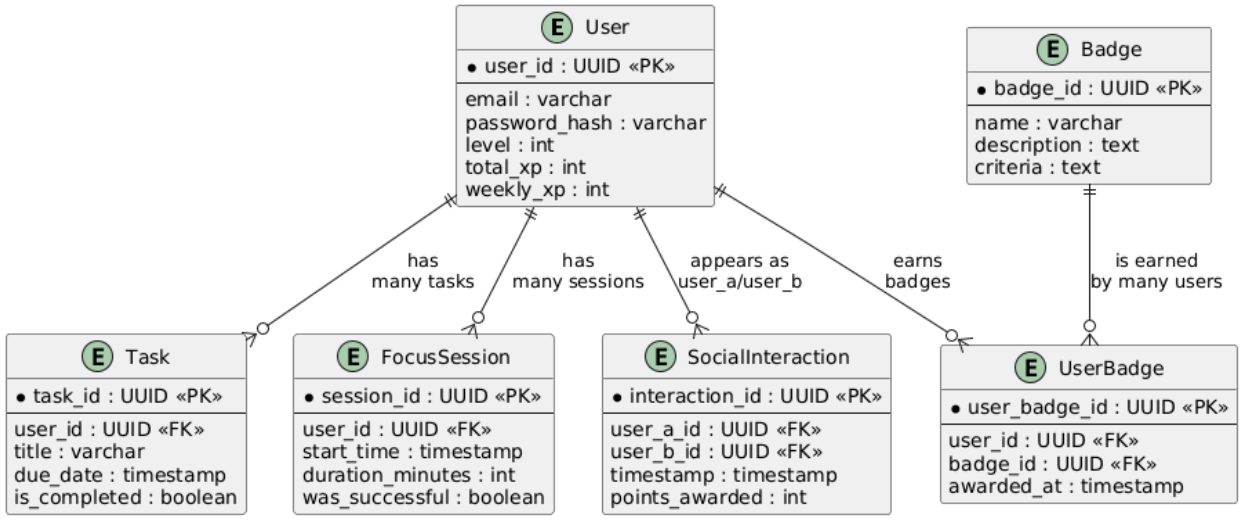
- **Client (Frontend):** Native Android Application (Kotlin). Responsible for UI rendering and user input.
- **Server (Backend):** Python Django Application. Responsible for business logic, gamification rules, and database management.
- **Database:** SQLite (Development) / PostgreSQL (Production). Stores all persistent data.
- **Communication:** RESTful API (JSON over HTTPS).



2. Database Design

2.1 Entity-Relationship Diagram

This diagram illustrates the logical relationships between data entities.



2.2 Database Schema & SQL Scripts

The following SQL statements represent the schema design for the core tables. Although Django ORM handles migration automatically, these scripts define the underlying structure.

(1) Users Table Stores user credentials and gamification stats.

```
CREATE TABLE users_user (
    id UUID PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    username VARCHAR(150),
    level INT DEFAULT 1,
    current_xp INT DEFAULT 0,
    total_credits INT DEFAULT 0,
    joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

(2) Tasks Table Stores the to-do list items linked to a user.

```
CREATE TABLE productivity_task (
    id UUID PRIMARY KEY,
    user_id UUID REFERENCES users_user(id) ON DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    description TEXT,
```

```
    due_date TIMESTAMP,
    is_completed BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

(3) Focus Sessions Table Records study sessions for gamification calculation.

```
CREATE TABLE productivity_focussession (
    id UUID PRIMARY KEY,
    user_id UUID REFERENCES users_user(id) ON DELETE CASCADE,
    start_time TIMESTAMP NOT NULL,
    duration_minutes INT NOT NULL,
    status VARCHAR(20)
);
```

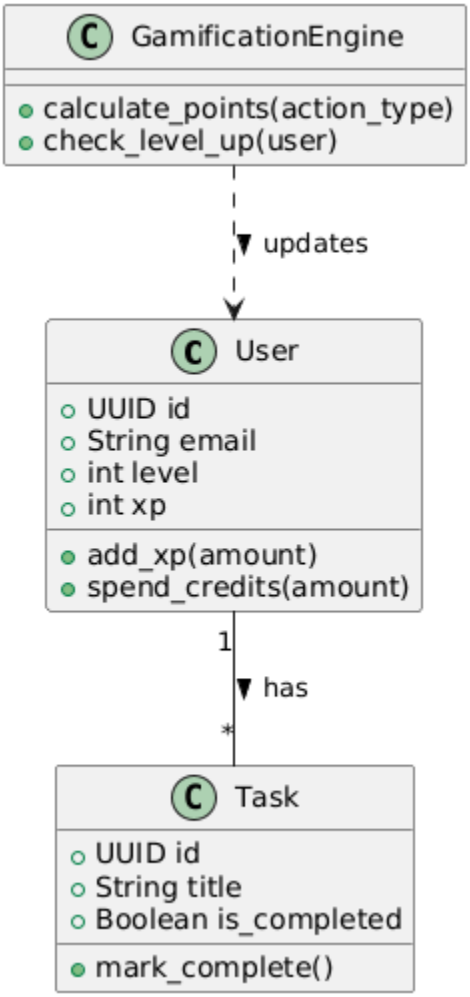
(4) Social Interactions Table Stores the Bluetooth-confirmed interactions.

```
CREATE TABLE social_interaction (
    id UUID PRIMARY KEY,
    initiator_id UUID REFERENCES users_user(id),
    receiver_id UUID REFERENCES users_user(id),
    confirmed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    points_awarded INT DEFAULT 0,
    CONSTRAINT unique_daily_interaction UNIQUE (initiator_id, receiver_id,
DATE(confirmed_at))
);
```

3. Component Design (UML Class Diagram)

3.1 Backend Class Diagram (Django Models)

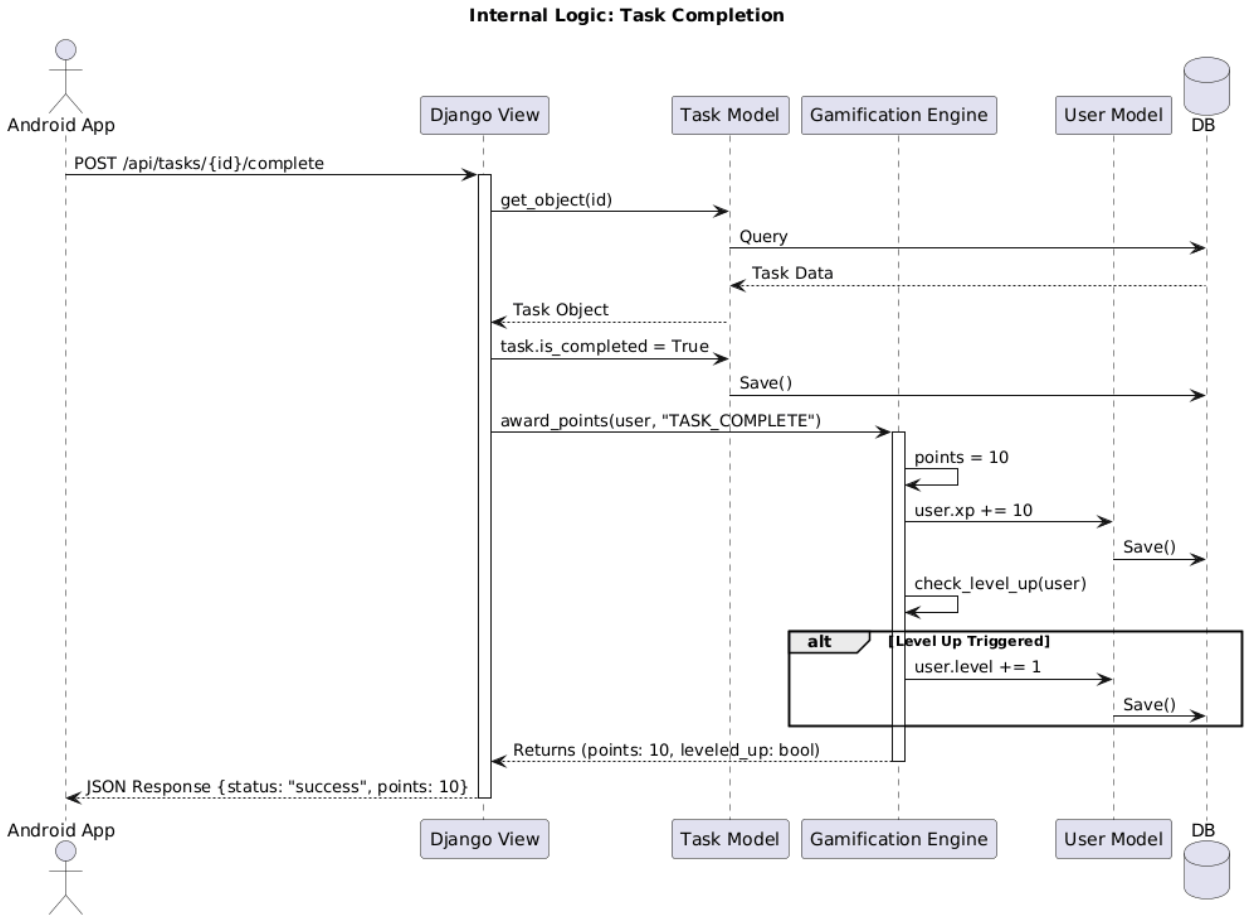
This diagram shows how the Python classes map to the database.



4. Sequence Diagrams

4.1 Core Logic: Completing a Task & Earning XP

This sequence details the backend logic when a user finishes a task.



API Interface Design

HTTP Method	Endpoint	Description	Request Body	Response
POST	/api/auth/register/	Create new user	{email, password}	{token, user_id}
POST	/api/auth/login/	Login user	{email, password}	{token}
GET	/api/tasks/	Get user's tasks	-	[{id, title, status}...]
POST	/api/tasks/	Create task	{title, due_date}	{id, title}
PATCH	/api/tasks/{id}/	Update task status	{is_completed: true}	{points_earned: 10}
POST	/api/social/confirm/	Log interaction	{target_user_id}	{status: "confirmed"}